INFO-F-302, Cours d'Informatique Fondamentale

Emmanuel Filiot et Jean-François Raskin
Département d'Informatique
Faculté des Sciences
Université Libre de Bruxelles

Année académique 2026-2027

Le Problème SAT

3 Rappels

Satisfaisabilité

Soit P un ensemble de propositions. Une formule ϕ de la logique propositionnelle sur P est *satisfaisable* si

3 Rappels

Satisfaisabilité

Soit P un ensemble de propositions. Une formule ϕ de la logique propositionnelle sur P est *satisfaisable* si il existe une interprétation $V : X \to \{1, 0\}$ telle que $V \models \phi$.

3 Rappels

Satisfaisabilité

Soit P un ensemble de propositions. Une formule ϕ de la logique propositionnelle sur P est *satisfaisable* si il existe une interprétation $V : X \to \{1, 0\}$ telle que $V \models \phi$.

Dans ce chapitre, nous allons utiliser des solveurs SAT, qui permettent de tester la satisfiabilité de formules représentées comme des **ensembles de clauses**.

4 Littéraux et Clauses

Définition

- Un littéral est une variable x ou sa négation $\neg x$
- Une clause est une disjonction de littéraux $\ell_1 \vee \cdots \vee \ell_n$. Elle est satisfaite par une valuation V s'il existe i tel que $V(\ell_i) = 1$. Par extension, on appelle $\bot$ la *clause vide*, qui est insatisfaisable.

Donc, p et $\neg p$ sont des littéraux. $x \vee \neg y \vee r$ est une clause, mais pas $x \wedge y$.

4 Littéraux et Clauses

Définition

- Un littéral est une variable x ou sa négation $\neg x$
- Une clause est une disjonction de littéraux $\ell_1 \vee \cdots \vee \ell_n$. Elle est satisfaite par une valuation V s'il existe i tel que $V(\ell_i) = 1$. Par extension, on appelle $\bot$ la *clause vide*, qui est insatisfaisable.

Donc, p et $\neg p$ sont des littéraux. $x \vee \neg y \vee r$ est une clause, mais pas $x \wedge y$.

Satisfaction d'ensemble de clauses

Un ensemble de clauses $A = \{C_1, \ldots, C_n\}$ est satisfait par une valuation V, noté $V \models A$, si pour tout i, $V \models C_i$. En particulier, tout valuation satisfait l'ensemble vide $A = \varnothing$.

Lien avec les formes normales

Une formule est en *forme normale conjonctive* (FNC) ssi c'est une conjonction de disjonctions de littéraux, de la forme : de la forme :

$$\bigwedge_i (\bigvee_j (\neg) x_{i,j})$$

Une formule est en *forme normale disjonctive* (FND) ssi c'est une disjonction de conjonctions de littéraux, de la forme : de la forme :

$$\bigvee_i (\bigwedge_j (\neg) x_{i,j})$$

Lien avec les formes normales

Une formule est en *forme normale conjonctive* (FNC) ssi c'est une conjonction de disjonctions de littéraux, de la forme : de la forme :

$$\bigwedge_i (\bigvee_j (\neg) x_{i,j})$$

Une formule est en *forme normale disjonctive* (FND) ssi c'est une disjonction de conjonctions de littéraux, de la forme : de la forme :

$$\bigvee_i (\bigwedge_j (\neg) x_{i,j})$$

Lemme

Tout ensemble non-vide de clauses $A = \{C_1, \ldots, C_n\}$ *est équivalent à la formule en FNC* $\phi_A = \bigwedge_{i=1}^{n} C_i$, *au sens où pour toute valuation,* $V \models A$ *ssi* $V \models \phi_A$.

$\Rightarrow$ Pour mettre une formule sous forme de clauses, il suffit de la mettre en FNC !

6 Mise sous FNC et FND

On utilise les transformations successives suivantes pour obtenir les formes normales :

1. élimination des connecteurs $\rightarrow$ et $\leftrightarrow$ grâce aux équivalences suivantes :

$$(\phi \rightarrow \psi) \equiv (\neg\phi \vee \psi)$$
$$(\phi \leftrightarrow \psi) \equiv (\neg\phi \vee \psi) \wedge (\phi \vee \neg\psi)$$

2. entrer les négations le plus à l'intérieur possible :

$$\neg(\phi \wedge \psi) \equiv (\neg\phi \vee \neg\psi) \qquad \neg\neg\phi \equiv \phi$$
$$\neg(\phi \vee \psi) \equiv (\neg\phi \wedge \neg\psi)$$

3. utilisation des distributivité de $\vee$ et $\wedge$:

$$(\phi \vee (\psi \wedge \chi)) \equiv (\phi \vee \psi) \wedge (\phi \vee \chi) \text{ (pour mise sous FNC)}$$
$$(\phi \wedge (\psi \vee \chi)) \equiv (\phi \wedge \psi) \vee (\phi \wedge \chi) \text{ (pour mise sous FND)}$$

Mise en FNC de $\neg(x \leftrightarrow (y \rightarrow r))$

Mise en FNC de $\neg(x \leftrightarrow (y \rightarrow r))$

1. on retire les équivalences et implications :

$$\neg((\neg x \vee \neg y \vee r) \wedge (\neg(\neg y \vee r) \vee x))$$

2. on pousse les négations à l'intérieur :

$$(x \wedge y \wedge \neg r) \vee ((\neg y \vee r) \wedge \neg x)$$

3. on distribue :

$$((x \wedge y \wedge \neg r) \vee (\neg y \vee r)) \wedge ((x \wedge y \wedge \neg r) \vee \neg x)$$

4. et encore :

$$(x \vee \neg y \vee r) \wedge (y \vee \neg y \vee r) \wedge (\neg r \vee \neg y \vee r) \wedge (x \vee \neg x) \wedge (y \vee \neg x) \wedge (\neg r \vee \neg x)$$

5. on retire les formules équivalentes à $\top$:

$$(x \vee \neg y \vee r) \wedge (y \vee \neg x) \wedge (\neg r \vee \neg x)$$

9 Exercice

Mettre les formules suivantes sous FNC :

1. $\neg(x \rightarrow q)$
2. $(x \wedge q) \vee r$
3. $x \wedge (y \vee (r \wedge t))$
4. $\neg(x \wedge ((r \vee q) \wedge s))$
5. $\neg(x \wedge (r \leftrightarrow q))$
6. $x \leftrightarrow (y \vee r)$
7. $(x_1 \wedge q_1) \vee (x_2 \wedge q_2) \vee (x_3 \wedge q_3)$
8. généraliser aux formules de la forme $(x_1 \wedge q_1) \vee (x_2 \wedge q_2) \vee \cdots \vee (x_n \wedge q_n)$ pour tout $n \geq 0$

10 Problème SAT

Définition

ENTREE Un ensemble de clauses S.
SORTIE Est-ce que S est **sat**isfaisable ?

Dans le cas où S est satisfaisable, on aimerait que l'algorithme nous retourne une valuation V qui satisfait S.

11 Solveurs SAT

- Un solveur SAT est un programme qui *décide* le problème SAT, i.e. qu'étant donné un ensemble de clauses S, il retourne le Booléen `true` si et seulement si S est satisfaisable.
- De plus, si S satisfaisable, une interprétation qui le satisfait est retournée.

12 Motivations

- Beaucoup de problèmes s'expriment naturellement par des formules en FNC, et donc sous formes d'ensemble de clauses
- Par exemple, un problème de planification où l'on a beaucoup de contraintes (une conjonction) et chaque contrainte représente plusieurs alternatives possibles (disjonction).

12 Motivations

- Beaucoup de problèmes s'expriment naturellement par des formules en FNC, et donc sous formes d'ensemble de clauses
- Par exemple, un problème de planification où l'on a beaucoup de contraintes (une conjonction) et chaque contrainte représente plusieurs alternatives possibles (disjonction).
- Les problèmes de la classe NP (http://www.nada.kth.se/~viggo/problemlist/) se réduisent (i.e. se "transforment") tous au problème SAT en temps polynomial.

12 Motivations

- Beaucoup de problèmes s'expriment naturellement par des formules en FNC, et donc sous formes d'ensemble de clauses
- Par exemple, un problème de planification où l'on a beaucoup de contraintes (une conjonction) et chaque contrainte représente plusieurs alternatives possibles (disjonction).
- Les problèmes de la classe NP (http://www.nada.kth.se/~viggo/problemlist/) se réduisent (i.e. se "transforment") tous au problème SAT en temps polynomial.
- **Motivation principale :** Plutôt que d'avoir un algorithme pour chaque problème, écrire un bon algorithme pour SAT et transformer automatiquement chaque instance du problème de départ en une instance de SAT équivalente.

12 Motivations

- Beaucoup de problèmes s'expriment naturellement par des formules en FNC, et donc sous formes d'ensemble de clauses
- Par exemple, un problème de planification où l'on a beaucoup de contraintes (une conjonction) et chaque contrainte représente plusieurs alternatives possibles (disjonction).
- Les problèmes de la classe NP (http://www.nada.kth.se/~viggo/problemlist/) se réduisent (i.e. se "transforment") tous au problème SAT en temps polynomial.
- **Motivation principale :** Plutôt que d'avoir un algorithme pour chaque problème, écrire un bon algorithme pour SAT et transformer automatiquement chaque instance du problème de départ en une instance de SAT équivalente.
- Si la transformation (aussi appelée réduction) se fait dans un temps raisonnable (polynomiale), alors l'utilisation d'un solveur SAT donne parfois d'aussi bons résultats qu'un algorithme dédié au problème de départ.

13 Histoire des Solveurs SAT

- **Davis & Putnam, 1960 ; Davis, Longemann & Loveland, 1962** Algorithme **DPLL**.
- **Tseitin, 1966** DPLL a une complexité en temps exponentiel au pire cas
- **Cook 1971** SAT est NP-complet
- Seconde Génération de solveurs SAT basés sur DPLL : Posit [1995], SATO [1997], GRASP [1999].
- Compétition SAT depuis 2002 : http://www.satcompetition.org/
- Solveurs SAT très efficaces : Chaff [2001], BerkMin [2002], zChaff [2004], MiniSAT [2005] ... Regroupent différentes techniques d'optimisation telles que l'apprentissage, des structures de données efficaces pour stocker les clauses, des heuristiques etc ...

14 Notations pour les grandes disjonctions et conjonctions

Définition

Disjonction et conjonction

- On appelera *disjonction* une formule de la forme $\phi_1 \vee \phi_2 \vee \cdots \vee \phi_n$, où $n \geq 1$. Attention lorsque $n = 1$, la disjonction est réduite à la seule formule ϕ_1. On utilisera l'abbréviation

$$\bigvee_{i=1}^{n} \phi_i$$

  pour noter cette discjontion.

- On appelera *conjonction* une formule de la forme $\phi_1 \wedge \phi_2 \wedge \cdots \wedge \phi_n$, où $n \geq 1$. Attention lorsque $n = 1$, la conjonction est réduite à la seule formule ϕ_1. On utilisera l'abbréviation

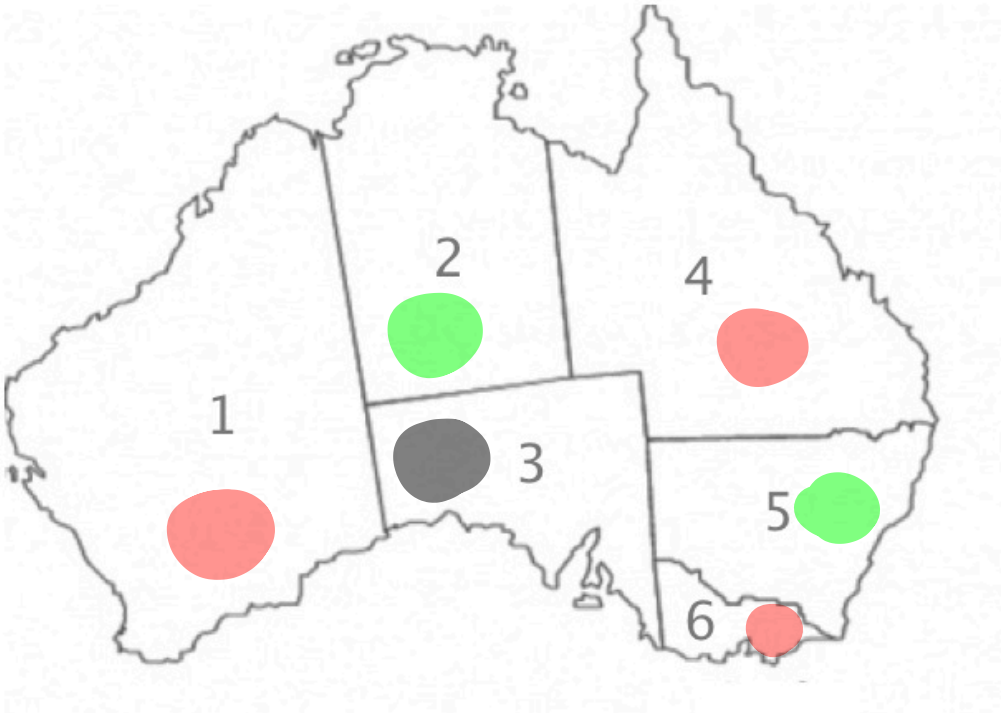
$$\bigwedge_{i=1}^{n} \phi_i$$

15 Modélisation

Nous allons maintenant nous intéresser à la modélisation de plusieurs problèmes en logique propositionnelle :

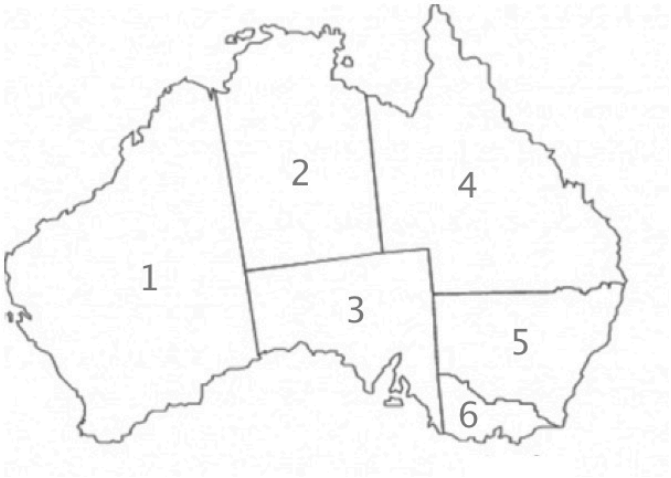
- le coloriage de cartes
- le problème des 8 reines
- le jeu Sudoku

16 Coloriage de Cartes



- colorier chaque région de telle sorte que deux régions ayant une frontière commune n'aient jamais la même couleur
- but : minimiser le nombre de couleurs

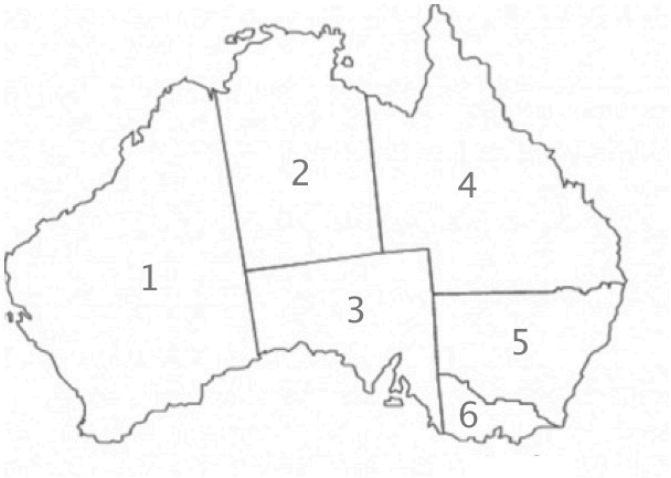
17 Coloriage de Cartes - Modélisation



Questions

- combien faut-il de couleur minimum pour la carte du dessus ?

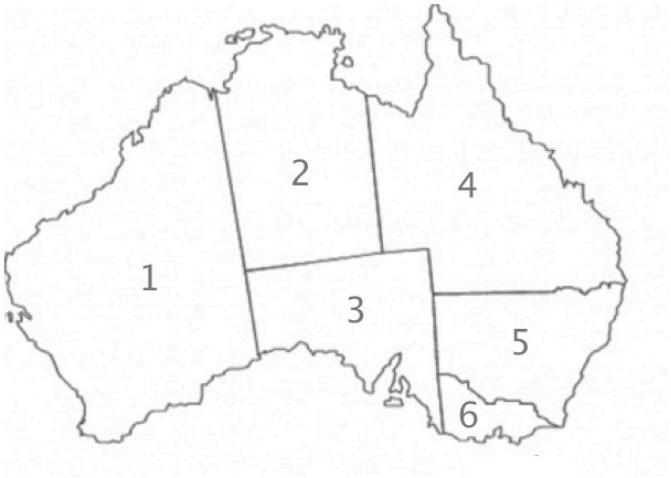
17 Coloriage de Cartes - Modélisation

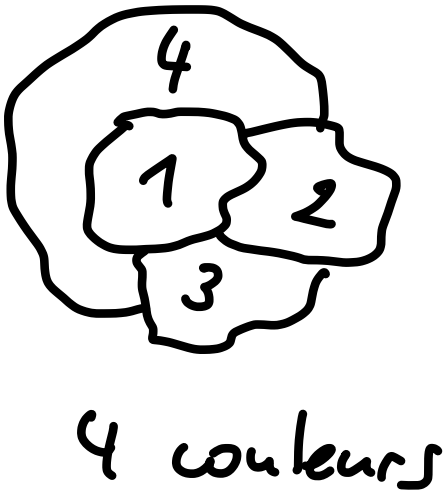


Questions

- combien faut-il de couleur minimum pour la carte du dessus ? 3 couleurs

17 Coloriage de Cartes - Modélisation

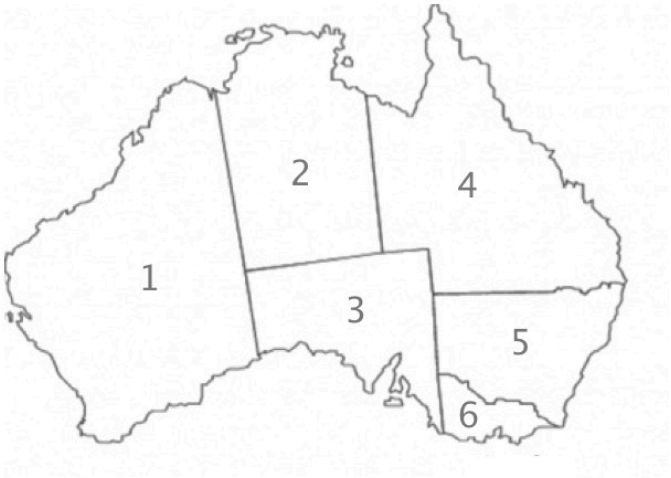




Questions

- combien faut-il de couleur minimum pour la carte du dessus ? 3 couleurs
- pourriez-vous donner une carte où il faut au moins 4 couleurs ?

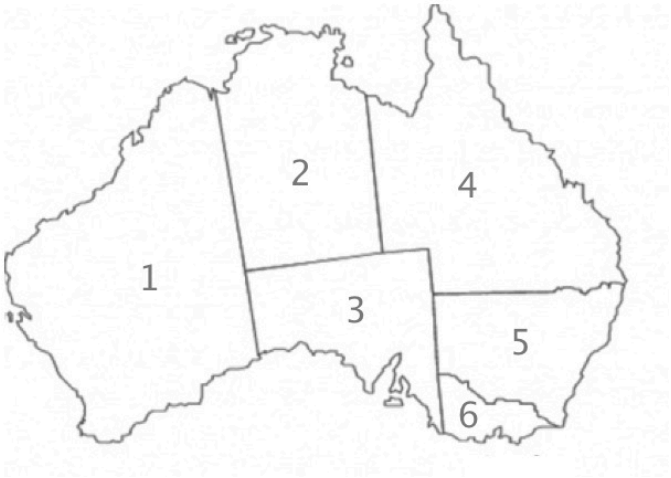
17 Coloriage de Cartes - Modélisation



Questions

- combien faut-il de couleur minimum pour la carte du dessus ? 3 couleurs
- pourriez-vous donner une carte où il faut au moins 4 couleurs ?
- et 5 couleurs ?

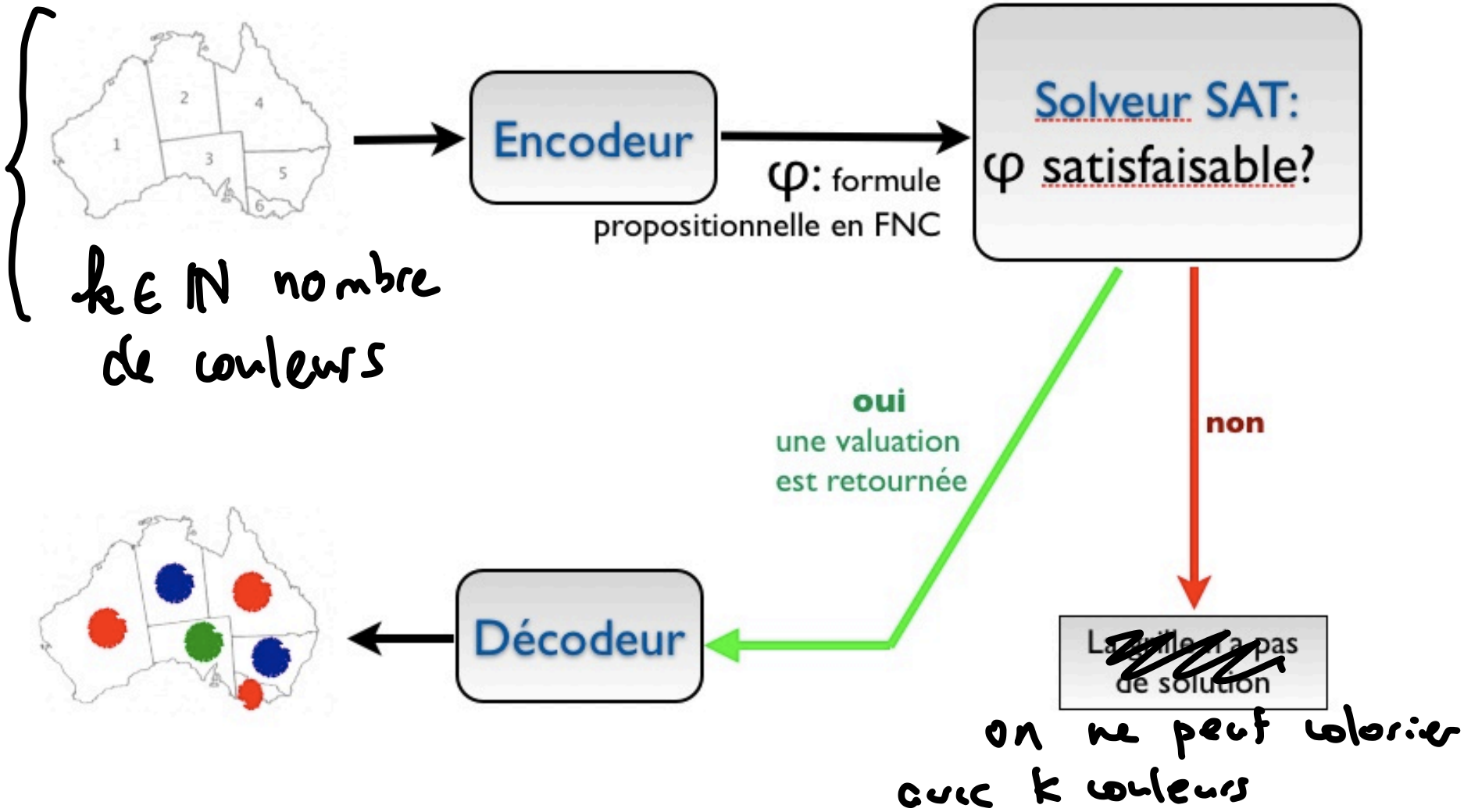
17 Coloriage de Cartes - Modélisation



Questions

- combien faut-il de couleur minimum pour la carte du dessus ? 3 couleurs
- pourriez-vous donner une carte où il faut au moins 4 couleurs ?
- et 5 couleurs ?
- **Remarque :** lorsque les régions ne sont pas discontinues (i.e. qu'une région est nécessairement en un seul morceau), alors 4 couleurs suffisent toujours (*Théorème de 4 couleurs*)

Modélisation : Rappel



Coloriage de Cartes - Modélisation

Choix des Propositions

- fixons un nombre de couleurs (3) et exprimons l'existence d'un coloriage avec 3 couleurs en logique propositionnelle.
- la première étape
- ici, l'ensemble des régions est $\{1, 2, 3, 4, 5, 6\}$ et l'ensemble des couleurs est $\{R, V, B\}$. Pour chaque région r et chaque couleur c, nous introduisons la proposition "la région r est coloriée avec la couleur c", notée $x_{r,c}$. On a donc l'ensemble des variables suivant :

$$X = \{x_{r,c} \mid r \in \{1, \ldots, 6\},\ c \in \{R, V, B\}\}$$

$x_{1,R} = 1$

$x_{1,B} = 1$

$x_{1,V} = 1$

$x_{2,R} = 1$

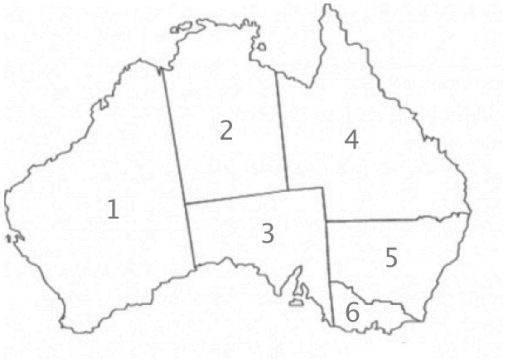
$x_{2,B} = 1$

$x_{2,V} = 1$

$\vdots$

On va vouloir exclure ce type de solution en exprimant qu'il existe au plus une couleur par région

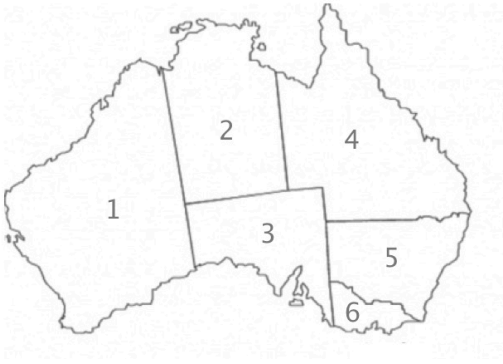
Coloriage de Cartes - Modélisation



Expression de la contrainte de bon coloriage

- les régions voisines doivent avoir des couleurs différentes, ici les régions voisines sont données par l'ensemble des paires $\mathit{Voisines} = \{(1,2),(1,3),(2,4),(3,4),(4,5),(3,5),(5,6),(3,6)\}$.

Coloriage de Cartes - Modélisation

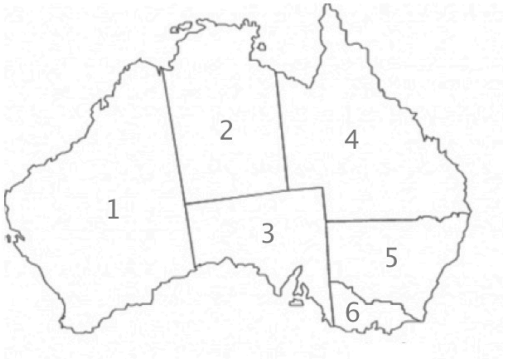


Expression de la contrainte de bon coloriage

- les régions voisines doivent avoir des couleurs différentes, ici les régions voisines sont données par l'ensemble des paires $\mathit{Voisines} = \{(1,2),(1,3),(2,4),(3,4),(4,5),(3,5),(5,6),(3,6)\}$.
- comment exprimer que 1 et 2 ont des couleurs différentes ?

$\neg(X_{1,R} \wedge X_{2,R})$ "1 et 2 n'ont pas la couleur rouge"

Coloriage de Cartes - Modélisation



Expression de la contrainte de bon coloriage

- les régions voisines doivent avoir des couleurs différentes, ici les régions voisines sont données par l'ensemble des paires $\mathit{Voisines} = \{(1,2),(1,3),(2,4),(3,4),(4,5),(3,5),(5,6),(3,6)\}$.
- comment exprimer que 1 et 2 ont des couleurs différentes ?

$$\begin{aligned} & \neg((x_{1,V} \wedge x_{2,V}) \vee (x_{1,B} \wedge x_{2,B}) \vee (x_{1,R} \wedge x_{2,R}) \\ \equiv\ & (\neg x_{1,V} \vee \neg x_{2,V}) \wedge (\neg x_{1,B} \vee \neg x_{2,B}) \wedge (\neg x_{1,R} \vee \neg x_{2,R}) \\ \equiv\ & \bigwedge_{c \in \{R,V,B\}} \neg x_{1,c} \vee \neg x_{2,c} \end{aligned}$$

Coloriage de Cartes - Modélisation



Expression de la contrainte de bon coloriage

- Il faut exprimer la contrainte pour toutes les paires voisines de $\mathit{Voisines} = \{(1,2),(1,3),(2,4),(3,4),(4,5),(3,5),(5,6),(3,6)\}$:

$$
\begin{aligned}
& (\neg x_{1,V} \vee \neg x_{2,V}) \wedge (\neg x_{1,B} \vee \neg x_{2,B}) \wedge (\neg x_{1,R} \vee \neg x_{2,R}) \\
\wedge\ & (\neg x_{1,V} \vee \neg x_{3,V}) \wedge (\neg x_{1,B} \vee \neg x_{3,B}) \wedge (\neg x_{1,R} \vee \neg x_{3,R}) \\
\wedge\ & (\neg x_{2,V} \vee \neg x_{4,V}) \wedge (\neg x_{2,B} \vee \neg x_{4,B}) \wedge (\neg x_{2,R} \vee \neg x_{4,R}) \\
\wedge\ & (\neg x_{3,V} \vee \neg x_{4,V}) \wedge (\neg x_{3,B} \vee \neg x_{4,B}) \wedge (\neg x_{3,R} \vee \neg x_{4,R}) \\
\wedge\ & (\neg x_{4,V} \vee \neg x_{5,V}) \wedge (\neg x_{4,B} \vee \neg x_{5,B}) \wedge (\neg x_{4,R} \vee \neg x_{5,R}) \\
\wedge\ & (\neg x_{3,V} \vee \neg x_{5,V}) \wedge (\neg x_{3,B} \vee \neg x_{5,B}) \wedge (\neg x_{3,R} \vee \neg x_{5,R}) \\
\wedge\ & (\neg x_{5,V} \vee \neg x_{6,V}) \wedge (\neg x_{5,B} \vee \neg x_{6,B}) \wedge (\neg x_{5,R} \vee \neg x_{6,R}) \\
\wedge\ & (\neg x_{3,V} \vee \neg x_{6,V}) \wedge (\neg x_{3,B} \vee \neg x_{6,B}) \wedge (\neg x_{3,R} \vee \neg x_{6,R})
\end{aligned}
$$

Coloriage de Cartes - Modélisation



Expression de la contrainte de bon coloriage

- Il faut exprimer la contrainte pour toutes les paires voisines de $\mathit{Voisines} = \{(1,2),(1,3),(2,4),(3,4),(4,5),(3,5),(5,6),(3,6)\}$:

$$
\begin{aligned}
& (\textstyle\bigwedge_{c\in\{R,V,B\}} \neg x_{1,c} \vee \neg x_{2,c}) \\
\wedge\ & (\textstyle\bigwedge_{c\in\{R,V,B\}} \neg x_{1,c} \vee \neg x_{3,c}) \\
\wedge\ & (\textstyle\bigwedge_{c\in\{R,V,B\}} \neg x_{2,c} \vee \neg x_{4,c}) \\
\wedge\ & (\textstyle\bigwedge_{c\in\{R,V,B\}} \neg x_{3,c} \vee \neg x_{4,c}) \\
\wedge\ & (\textstyle\bigwedge_{c\in\{R,V,B\}} \neg x_{4,c} \vee \neg x_{5,c}) \\
\wedge\ & (\textstyle\bigwedge_{c\in\{R,V,B\}} \neg x_{3,c} \vee \neg x_{5,c}) \\
\wedge\ & (\textstyle\bigwedge_{c\in\{R,V,B\}} \neg x_{5,c} \vee \neg x_{6,c}) \\
\wedge\ & (\textstyle\bigwedge_{c\in\{R,V,B\}} \neg x_{3,c} \vee \neg x_{6,c})
\end{aligned}
$$

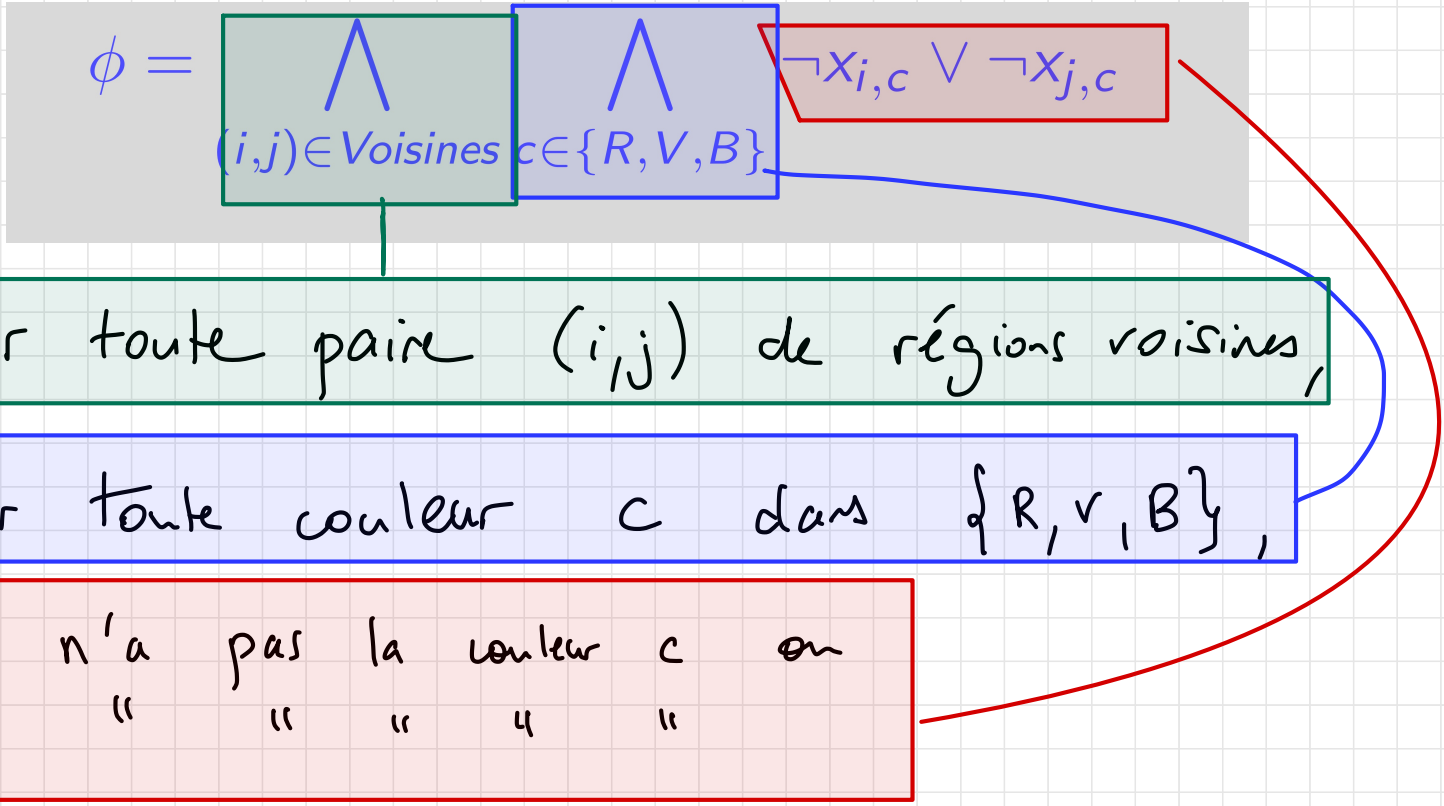
Coloriage de Cartes - Modélisation



Expression de la contrainte de bon coloriage

- Il faut exprimer la contrainte pour toutes les paires voisines de $\mathit{Voisines} = \{(1,2),(1,3),(2,4),(3,4),(4,5),(3,5),(5,6),(3,6)\}$:

$$\phi = \bigwedge_{(i,j)\in \mathit{Voisines}} \bigwedge_{c\in\{R,V,B\}} \neg x_{i,c} \vee \neg x_{j,c}$$


$\phi = \bigwedge_{(i,j) \in Voisines} \bigwedge_{c \in \{R,V,B\}} \neg x_{i,c} \vee \neg x_{j,c}$
Pour toute paire (i,j) de régions voisines,
Pour toute couleur c dans {R, V, B},
i n'a pas la couleur c ou
j " " " " "

" la région 1 possède au moins une couleur "

$$X_{1,R} \vee X_{1,B} \vee X_{1,V}$$

⇝ le faire pour toutes les régions.

Coloriage de Cartes - Modélisation

- La formule précédente est satisfaisable même si on met toutes les variables à 0
- Autrement dit, si on ne colorie rien, alors pas de risque de colorier deux régions voisines avec la même couleur !
- il manque la contrainte qu'il faut que chaque région soit coloriée

Expression de la contrainte d'existence d'une couleur pour chaque région

- On exprime que pour toute région, il existe une couleur qui la colorie :

$$\phi = \bigwedge_{r \in \{1,\ldots,6\}} \left(\bigvee_{c \in \{R,V,B\}} x_{r,c} \right)$$

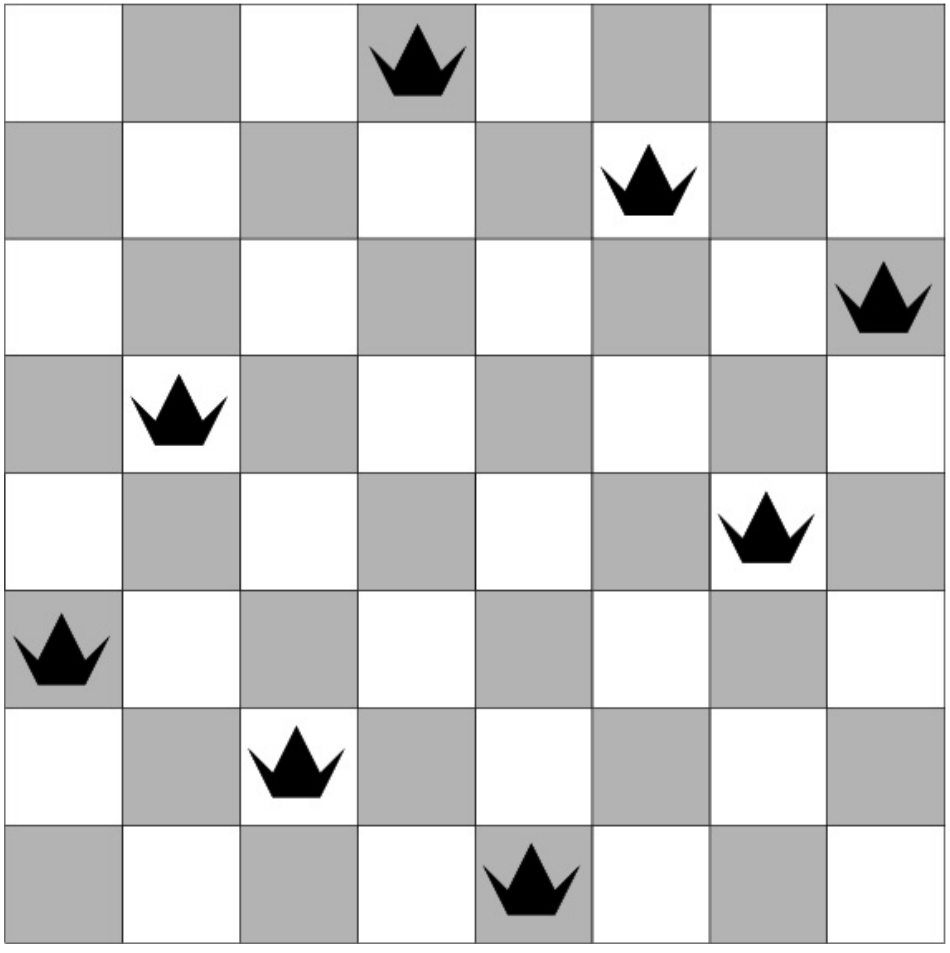
25 Coloriage de graphes

- On peut généraliser le problème précédent des cartes aux graphes.
- Un graphe est coloriable avec k couleurs s'il existe une fonction qui associe à chaque sommet une unique couleur, telle que les sommets adjacents n'ont pas la même couleur
- Soit un graphe (fini) $G = (S, A)$ avec S l'ensemble des sommets et $A \subseteq V \times V$ l'ensemble d'arêtes, et soit k le nombre de couleurs.
- Pour tout $s \in S$ et $c \in \{1, \ldots, k\}$, on pose $x_{s,c}$ la proposition "s est coloriée avec c".
- La formule

$$\Big(\bigwedge_{(u,u')\in A}\ \bigwedge_{1\leq c<c'\leq k} \neg x_{u,c} \vee \neg x_{u',c'}\Big) \wedge \bigwedge_{u\in S}\Big(\bigvee_{1\leq c\leq k} x_{u,c}\Big)$$

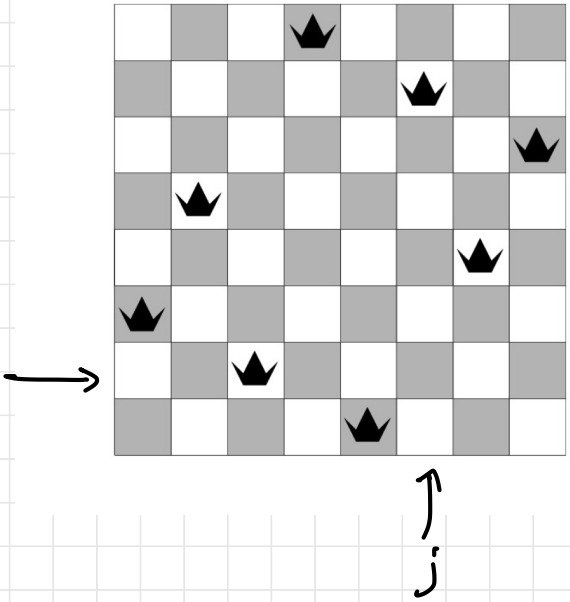
est satisfaisable ssi le graphe G est coloriable avec k couleurs au plus.

26 Problème des 8 reines (Au Tableau)



CHOIX DES VARIABLES

$X_{i,j}$: " il y a exactement une reine à la ligne i et colonne j "



CONTRAINTES

"au moins une reine par ligne"

Pour toute ligne $i \in \{1, \dots, 8\}$,

il existe une colonne $j \in \{1, \dots, 8\}$,

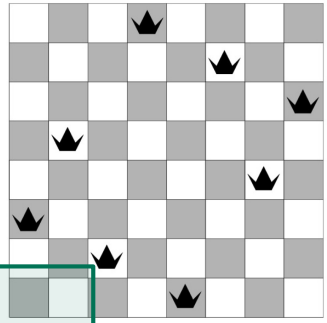
(i,j) contient une reine.

$$\varphi_1 = \bigwedge_{i \in \{1,\dots,8\}} \left(\bigvee_{j \in \{1,\dots,8\}} X_{i,j} \right)$$

"au plus une reine par ligne"

on exprime d'abord le contraire

⇝ il existe au moins deux reines en ligne 1, et plus précisément



il existe $j, k \in \{1, \dots, 8\}$ tels que $j \neq k$

et $(1, j)$ contient une reine et $(1, k)$ aussi

$$\bigvee_{\substack{j,k \in \{1,\dots,8\} \\ j \neq k}} X_{1,j} \wedge X_{1,k}$$

ON PREND LA NEGATION

$$\neg \left(\bigvee_{\substack{j,k \in \{1,\dots,8\} \\ j \neq k}} X_{i,j} \wedge X_{i,k} \right)$$

$$= \bigwedge_{\substack{j,k \in \{1,\dots,8\} \\ j \neq k}} (\neg X_{i,j} \vee \neg X_{i,k})$$

LE FAIRE POUR TOUTES LES LIGNES :

$$\varphi_2 = \bigwedge_{i \in \{1,\dots,8\}} \bigwedge_{\substack{j,k \in \{1,\dots,8\} \\ j \neq k}} (\neg X_{i,j} \vee \neg X_{i,k})$$

$\varphi_1 \wedge \varphi_2$ exprime que chaque ligne a exactement une reine

PAS D'ATTAQUE HORIZONTALE c'est φ_2 !

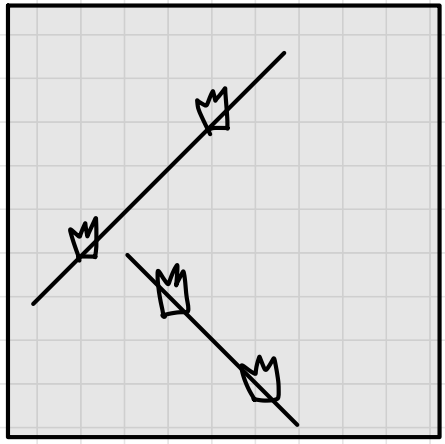
PAS D'ATTAQUE VERTICALE

$$\varphi_3 = \bigwedge_{i \in \{1,\dots,8\}} \bigwedge_{\substack{j,k \in \{1,\dots,8\} \\ j \neq k}} (\neg X_{j,i} \vee \neg X_{k,i})$$

PAS D'ATTAQUE DIAGONALE

Une case (i,j) est sur la même diagonale qu'une case (k,ℓ) ssi

$$|i-k| = |j-\ell|$$



On ne veut pas qu'il existe

(i,j) et (k,ℓ)

① sur la même diagonale

② avec des reines.

$$\neg\left(\bigvee_{\substack{(i,j)\in\{1,\dots,8\}^2\\(k,\ell)\in\{1,\dots,8\}^2\\|i-k|=|j-\ell|}} X_{i,j} \wedge X_{k,\ell}\right) = \bigwedge_{\substack{(i,j)\in\{1,\dots,8\}^2\\(k,\ell)\in\{1,\dots,8\}^2\\|i-k|=|j-\ell|}} (\neg X_{i,j} \vee \neg X_{k,\ell})$$

φ_4

ENSEMBLE FINAL DE CLAUSES

$\varphi = \varphi_1 \wedge \varphi_2 \wedge \varphi_3 \wedge \varphi_4$ est déjà en FNC

$\rightsquigarrow$ l'ensemble des clauses est donc donné par l'ensemble des disjonctions de φ.

8 Reines - Codage en PySAT (1)

- codons le problème dans la librairie PySAT
- PySAT est une librairie Python qui fournit une interface permettant d'uiliser différents solveurs SAT.

8 Reines - Codage en PySAT (1)

```
from pysat.solvers import Minisat22
from pysat.solvers import Glucose4
from pysat.formula import CNF
from pysat.formula import IDPool


nb = 8 # nombre de reines

affichage_sol = True # affichage d'une solution

vpool = IDPool(start_from=1) # pour le stockage des identifiants entiers des couples (i,j)

cnf = CNF()  # construction d'un objet formule en forme normale conjonctive (Conjunctive Normal Form)
```

- Pour PySAT, une variable booléenne est un entier positif, par exemple 7, et sa négation un entier négatif, par exemple, -7
- La classe `IDPool` permet de créer automatiquement un codage des objets qu'on veut manipuler, par exemple des paires d'entiers, par des entiers positifs
- `cnf` est un objet qui représente une formule en FNC. Chaque clause est une liste de litéraux, et une formule est une liste de clauses.

$$\bigwedge_{i \in \{1,\dots,8\}} \left(\bigvee_{j \in \{1,\dots,8\}} x_{i,j} \right)$$

8 Reines - Codage en PySAT (2)

Le code suivant ajoute un ensemble de clauses qui code la contrainte qu'il existe au moins une reine par ligne `i` :

```
print("Au moins un reine par ligne")

# au moins un reine par ligne
for i in range(nb):
    d = □
    for j in range(nb):
        d.append(vpool.id((i,j)))
    cnf.append(d)
```

- la variable `d` représente une disjonction de litéraux
- `vpool.id((i,j))` retourne un entier qui code la paire `(i,j)`, et l'ajoute à `vpool` si c'est la première fois que cette paire est rencontrée
- `cnf.append(d)` ajoute la clause `d` à l'ensemble des clauses

8 Reines - Codage en PySAT (3)

Le code suivant encode les contraintes qui expriment qu'il n'y a ni d'attaque en ligne ni d'attaque en colonne :

```
# pas d'attaque en ligne
print("Pas d'attaque en ligne")

for i in range(nb):
    for j in range(nb):
        for k in range(nb):
            if k > j:
                cnf.append([-vpool.id((i,j)),-vpool.id((i,k))])

# pas d'attaque en colonne
print("Pas d'attaque en colonne")

for i in range(nb):
    for j in range(nb):
        for k in range(nb):
            if k > j:
                cnf.append([-vpool.id((j,i)),-vpool.id((k,i))])
```

- `cnf.append([-vpool.id((i,j)),-vpool.id((i,k))])` ajoute une clause à deux éléments, en négation.

8 Reines - Codage en PySAT (4)

Ici on encode la contrainte "pas d'attaque diagonale" :

```
# pas d'attaque en diagonale

print("Pas d'attaque en diagnale")

# version lente
for i in range(nb):
    for j in range(nb):
        for k in range(nb):
            for l in range(nb):
                if abs(i-k)==abs(j-l) and i!=k:
                    cnf.append([-vpool.id((i,j)),-vpool.id((k,l))])
```

- Pour toute paire de cases `(i,j)` et `(k,l)`, le code `abs(i-k)==abs(j-l)` teste si elles sont sur la même diagonale
- ce code est lent quand `nb` est grand, on va voir une version optimisée.

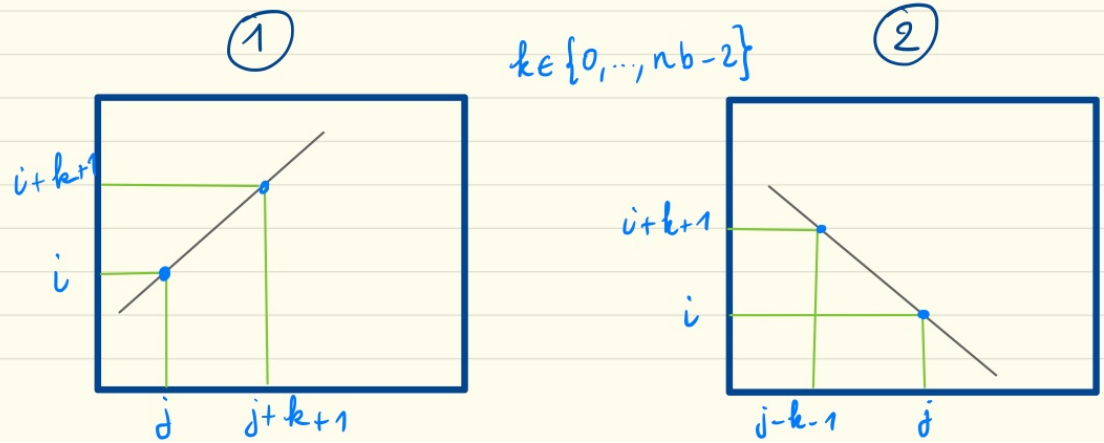
8 Reines - Codage en PySAT (5)

Version optimisée :

```
# version optimisee
for i in range(nb):
    for j in range(nb):
        for k in range(nb-1):
                if i+k+1<nb and j+k+1<nb:
                    cnf.append([-vpool.id((i,j)),-vpool.id((i+k+1,j+k+1))])


for i in range(nb):
    for j in range(nb):
        for k in range(nb-1):
                if i+k+1<nb and j-k-1>=0:
                    cnf.append([-vpool.id((i,j)),-vpool.id((i+k+1,j-k-1))])
```

Deux cas :



8 Reines - Codage en PySAT (6)

Lancement d'un solveur :

```
# phase de resolution

s = Minisat22(use_timer=True) # pour utiliser le solveur MiniSAT
# s = Glucose4(use_timer=True) # pour utiliser le solveur Glucose
s.append_formula(cnf.clauses, no_return=False)

print("Resolution...")
resultat = s.solve()
print("satisfaisable : " + str(resultat))

print("Temps de resolution : " + '{0:.2f}s'.format(s.time()))
```

- `s = Minisat22(use_timer=True)` crée un objet solveur MiniSAT (on pourrait utiliser un autre solveur)
- `use_timer=True` permet de compter le temps passé à la résolution
- `s.append_formula(cnf.clauses, no_return=False)` ajoute la formule au solveur
- `s.solve()` lance le solveur et retourne le résultat dans `resultat`
- `s.time()` retourne le temps passé à la résolution (depuis le dernier appel à la méthode `solve()`).

8 Reines - Codage en PySAT (7)

Remarque : La résolution est incrémentale : si on ajoute de nouvelles clauses à `s` et qu'on appelle une deuxième fois la méthode `solve()`, la résolution réutilise les calculs déjà faits lors du premier appel.

8 Reines - Codage en PySAT (8)

Affichage de la solution trouvée :

```
# affichage solution

if affichage_sol and resultat:

    print("\nVoici une solution: \n")

    for i in range(nb):
        for j in range(nb):
            if vpool.id((i,j)) in s.get_model():
                print("R ",end='')
            else:
                print("* ",end='')
        print("")
```

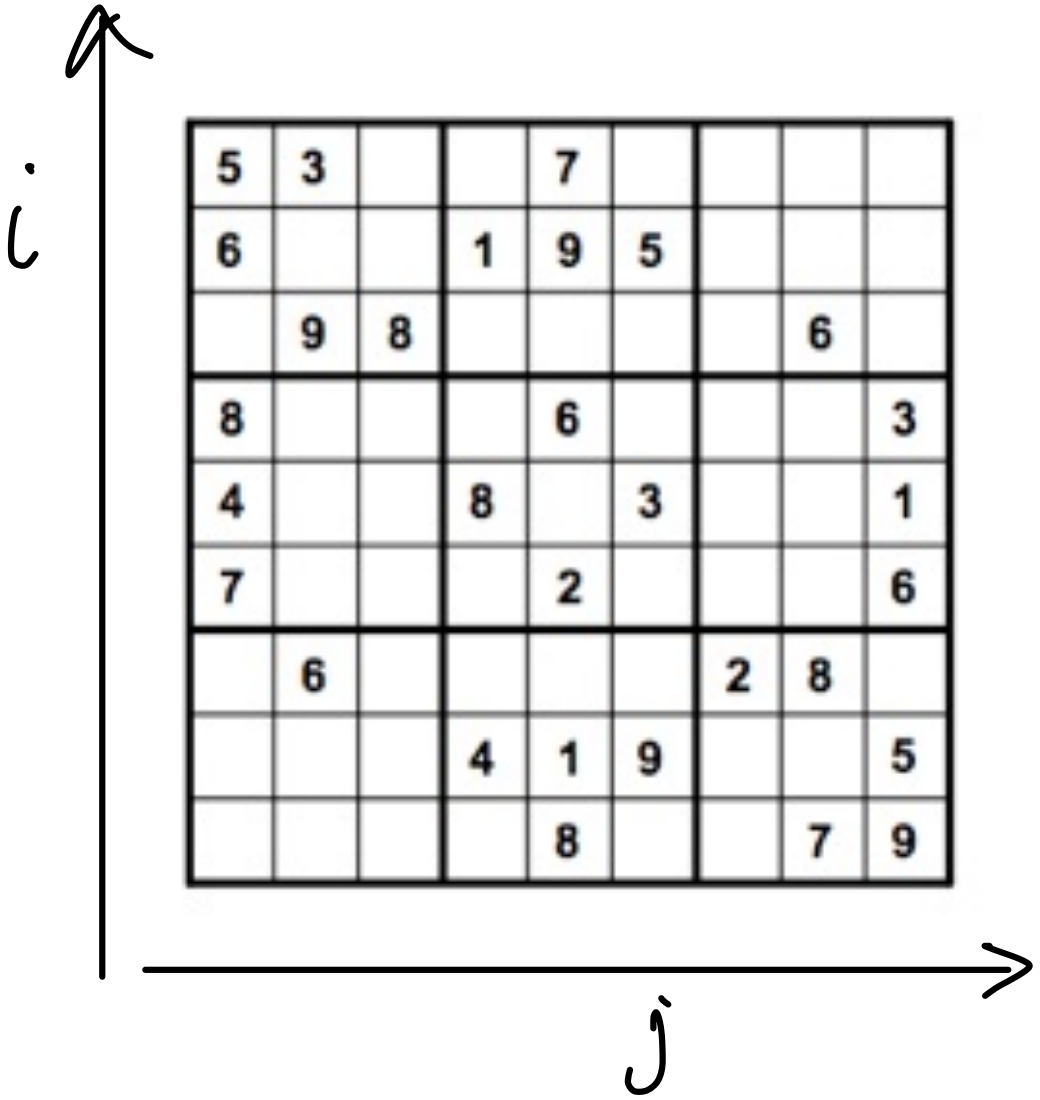
- `s.get_model()` retourne une interprétation qui satisfait la formule, sous forme de liste de litéraux.
- Par exemple, $V(1) = V(2) = 0$ et $V(3) = V(4) = V(5) = 1$ est représentée par la liste `[-1,-2,3,4,5]`.

36 Problème du Sudoku (Au tableau)

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

5	3	4	6	7	8	9	1	2
6	7	2	1	9	5	3	4	8
1	9	8	3	4	2	5	6	7
8	5	9	7	6	1	4	2	3
4	2	6	8	5	3	7	9	1
7	1	3	9	2	4	8	5	6
9	6	1	5	3	7	2	8	4
2	8	7	4	1	9	6	3	5
3	4	5	2	8	6	1	7	9

39 Problème du Sudoku (Au tableau)



5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

5	3	4	6	7	8	9	1	2
6	7	2	1	9	5	3	4	8
1	9	8	3	4	2	5	6	7
8	5	9	7	6	1	4	2	3
4	2	6	8	5	3	7	9	1
7	1	3	9	2	4	8	5	6
9	6	1	5	3	7	2	8	4
2	8	7	4	1	9	6	3	5
3	4	5	2	8	6	1	7	9

CHOIX DES VARIABLES

On cherche une fonction $f : \{1,\dots,9\}^2 \longrightarrow \{1,\dots,9\}$

coordonnées des cases — valeurs

On peut voir cette fonction comme une fonction

$$g : \{1,\dots,9\}^3 \longrightarrow \{0,1\}$$

définie par $g(i,j,v) = 1$ ssi $f(i,j) = v$

Donc on peut prendre une variable par triplet $(i,j,v) \in \{1,\dots,9\}$

$$X = \{ x_{i,j,v} \mid i,j,v \in \{1,\dots,9\} \}$$

Jamais deux valeurs différentes dans une même case

- Si la case $(5,7)$ contient 3 alors elle ne contient pas 6

$$\neg\left(x_{5,7,3} \wedge x_{5,7,6}\right) \equiv \neg x_{5,7,3} \vee \neg x_{5,7,6}$$
$$\equiv x_{5,7,3} \rightarrow \neg x_{5,7,6}$$

- La case $(5,7)$ ne contient pas deux valeurs différentes

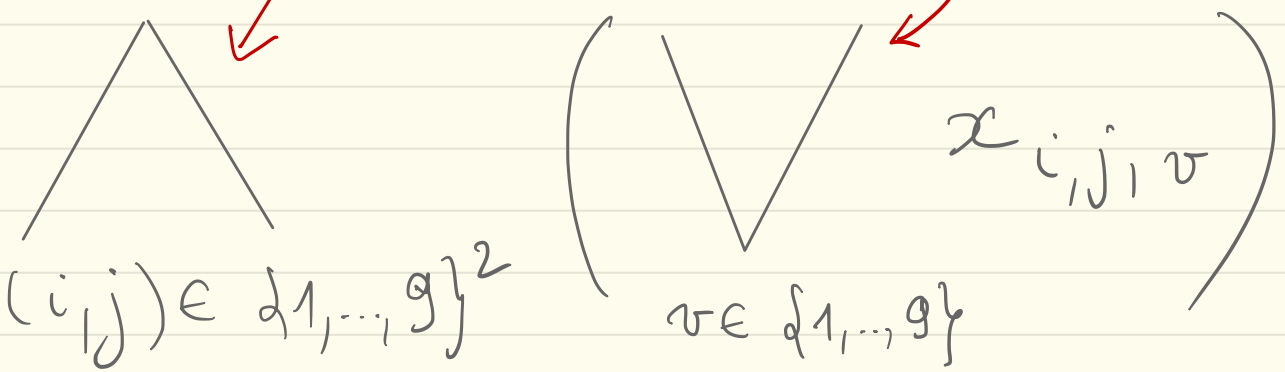
$$\bigwedge_{v=1}^{g}\ \bigwedge_{v'=v+1}^{g} \left(\neg x_{5,7,v} \vee \neg x_{5,7,v'}\right)$$

- Jamais deux $\neq$ dans une même case

$$\bigwedge_{(i,j)\in\{1,\dots,g\}^2}\ \bigwedge_{v=1}^{g}\ \bigwedge_{v'=v+1}^{g} \left(\neg x_{i,j,v} \vee \neg x_{i,j,v'}\right)$$

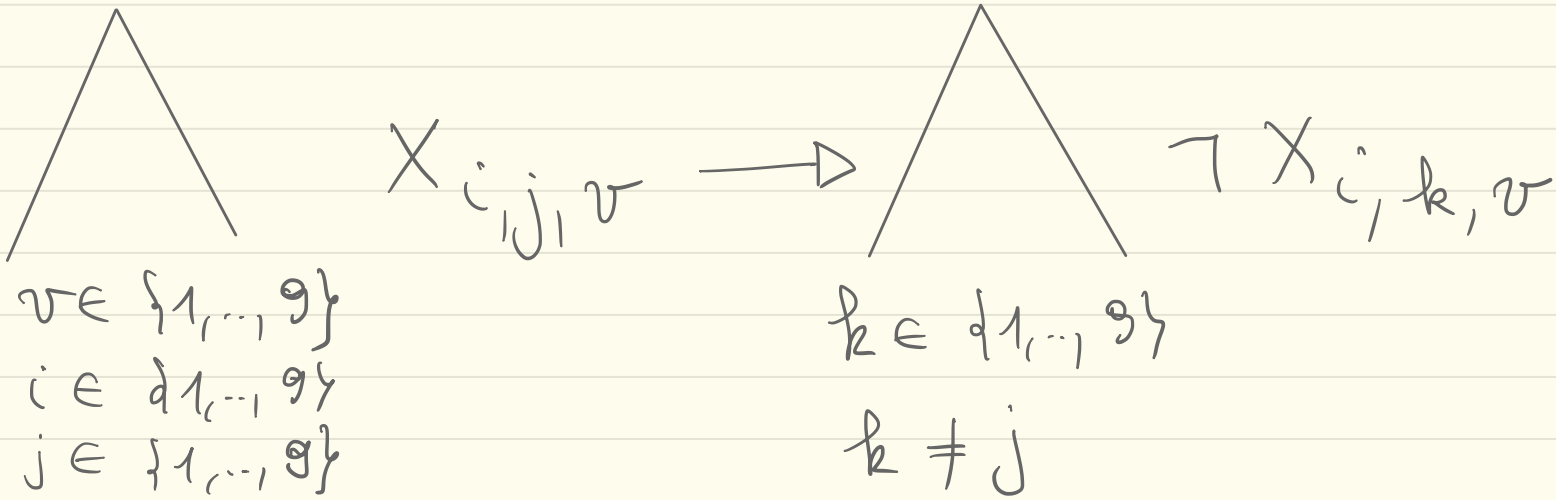
Au moins une valeur par case

Pour toute case (i,j), il existe une valeur v dans (i,j)



Jamais deux fois la même valeur sur une même ligne

Pour toute ligne i, pour toute colonne j, pour toute valeur v, si (i,j) contient v, alors pour toute colonne $k \neq j$, (i,k) ne contient pas v



$$\equiv \bigwedge_{\substack{v \in \{1,\dots,9\} \\ i \in \{1,\dots,9\} \\ j \in \{1,\dots,9\}}} \left(\neg X_{i,j,v} \vee \left(\bigwedge_{k \in \{1,\dots,9\},\ k \neq j} \neg X_{i,k,v} \right) \right)$$

$$\equiv \bigwedge_{\substack{v \in \{1,\dots,9\} \\ i \in \{1,\dots,9\}}} \bigwedge_{\substack{k \in \{1,\dots,9\} \\ j \in \{1,\dots,9\},\ k \neq j}} \left(\neg X_{i,j,v} \vee \neg X_{i,k,v} \right)$$

$$\equiv \bigwedge_{\substack{v \in \{1,\dots,9\} \\ i \in \{1,\dots,9\} \\ j \in \{1,\dots,9\}}} \left(\neg X_{i,j,v} \vee \neg X_{i,k,v} \right) \quad k \in \{j+1,\dots,9\}$$

Remarque : contrainte générale pour un sous-ensemble de cases.

Soit $C \subseteq \{1, \dots, 9\}^2$

Dans C, il n'existe pas deux fois la même valeur

$$\bigwedge_{(i,j) \in C} \left(\bigwedge_{\substack{(i',j') \in C \\ (i,j) \neq (i',j') \\ v \in \{1,\dots,9\}}} \left(\neg X_{i,j,v} \vee \neg X_{i',j',v} \right) \right)$$

Jamais deux fois la même valeur sur une même colonne

$$\bigwedge_{v \in \{1,\dots,9\}} \left(\neg X_{i,j,v} \vee \neg X_{i',j,v} \right)$$

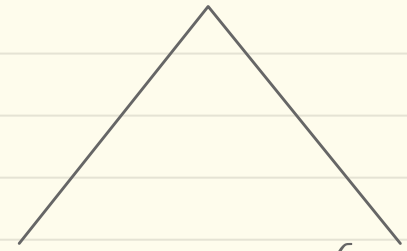
$v \in \{1,\dots,9\}$

$j \in \{1,\dots,9\}$

$i \in \{1,\dots,9\}$

$i' \in \{i+1,\dots,9\}$

Jamais deux fois la même valeur dans un même sous-carré 3×3



C sous-carré de la grille



$(i,j) \in C$

$(i',j') \in C$

$(i,j) \neq (i',j')$

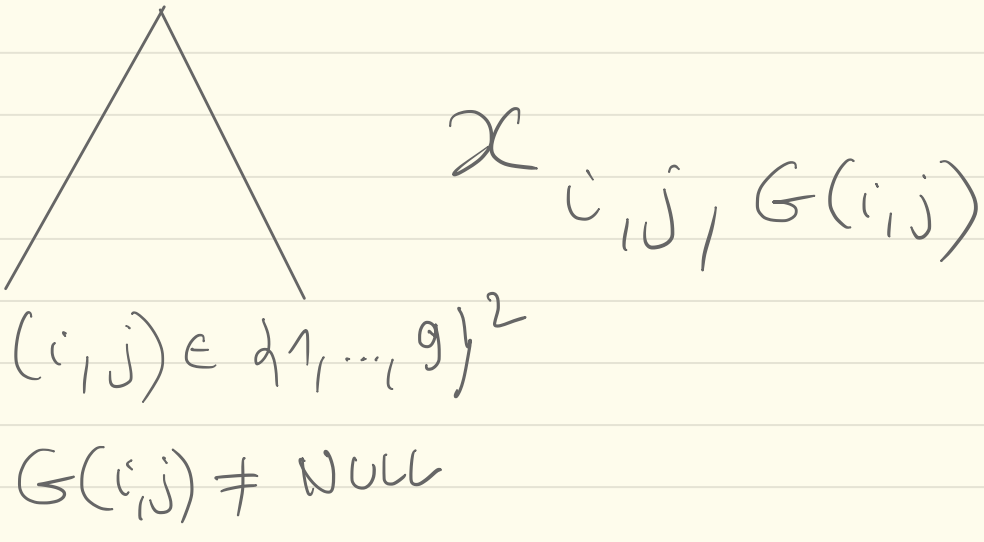
$v \in \{1, \dots, 9\}$

$$\left(\neg X_{i,j,v} \quad \vee \quad \neg X_{i',j',v} \right)$$

La grille est pré-remplie

En entrée on se donne une fonction $G : \{1,\dots,9\}^2 \longrightarrow \{1,\dots,9\} \cup \{NULL\}$

→ on doit exprimer l'existence des valeurs prédéfinies



Remarque : La contrainte du début "jamais deux valeurs différentes sur une même case" est inutile car impliquée par toutes les autres.

Néanmoins, lors des tests, il apparaît qu'elle permet de mieux guider le solveur et donc d'être plus efficace.

Comment choisir les propositions ?

Bien souvent, un problème de décision demande l'existence d'une relation binaire d'un ensemble fini A vers un ensemble fini B, qui satisfait certaines contraintes. Voici quelques exemples :

- **coloriage de graphes** : A est l'ensemble des sommets et B l'ensemble des couleurs.
- n **reines à placer sur un échiquier** $n \times n$: $A = \{1, \ldots, n\}^2$ est l'ensemble des cases et $B = \{0, 1\}$.
- **Sudoku** : $A = \{1, \ldots, 9\}^2$ est l'ensemble des cases et $B = \{1, \ldots, 9\}$ l'ensemble des valeurs.

Comment choisir les propositions ?

Bien souvent, un problème de décision demande l'existence d'une relation binaire d'un ensemble fini A vers un ensemble fini B, qui satisfait certaines contraintes. Voici quelques exemples :

- **coloriage de graphes** : A est l'ensemble des sommets et B l'ensemble des couleurs.
- n **reines à placer sur un échiquier** $n \times n$: $A = \{1, \ldots, n\}^2$ est l'ensemble des cases et $B = \{0, 1\}$.
- **Sudoku** : $A = \{1, \ldots, 9\}^2$ est l'ensemble des cases et $B = \{1, \ldots, 9\}$ l'ensemble des valeurs.

$\Rightarrow$ on peut alors tenter d'utiliser, pour tout $a \in A$ et $b \in B$, la proposition "a est en relation avec b", notée $x_{a,b}$.

Que faire si on veut que la relation soit une fonction ?

- dans les trois exemples précédents, on voudrait que la relation soit une fonction totale de A vers B (exactement une couleur par sommet, exactement une valeur 0 ou 1 par case de l'échiquier, exactement une valeur par case pour le Sudoku)
- il faut donc exprimer les contraintes suivantes :
  - **au moins une :** $\bigwedge_{a \in A} (\bigvee_{b \in B} x_{a,b})$
  - **au plus une :** $\bigwedge_{a \in A, b, b' \in B, b \neq b'} \neg x_{a,b} \vee \neg x_{a,b'}$

39 Transformation de Tseitin

- parfois, le problème qu'on cherche à résoudre ne s'exprime pas facilement par une formule en FNC
- Tranformation de Tseitin : ajouter des nouvelles variables et des équivalences

Exemple

Considérons un ensemble $X = \{x_1, \ldots, x_n\}$ de n variables. La formule

$$\phi_{\geq 2} = \bigvee_{1 \leq i < j \leq n} x_i \wedge x_j$$

est satisfaite dès lors qu'il existe au moins deux variables vraies.
Dans la transformation de Tseitin, on va introduire une nouvelle variable $y_{i,j}$ pour chaque $1 \leq i < j \leq n$ et on définit la formule suivante

$$\psi_{\geq 2} = \left(\bigvee_{1 \leq i < j \leq n} y_{i,k} \right) \wedge \left(\bigwedge_{1 \leq i < j \leq n} y_{i,j} \leftrightarrow (x_i \wedge x_j) \right)$$

40 Transformation de Tseitin

Exemple (Suite)

Ensuite, il reste à mettre $\psi_{\geq 2}$ en FNC :

- chaque équivalence $y_{i,j} \leftrightarrow (x_i \wedge x_j)$ est équivalente aux trois clauses

$$(\neg y_{i,j} \vee x_i) \wedge (\neg y_{i,j} \vee x_j) \wedge (\neg x_i \vee \neg x_j \vee y_{i,j})$$

- ceci permet de mettre $\psi_{\geq 2}$ en FNC :

$$\psi_{\geq 2} \equiv \left(\bigvee_{1 \leq i < j \leq n} y_{i,k} \right) \wedge \left(\bigwedge_{1 \leq i < j \leq n} (\neg y_{i,j} \vee x_i) \wedge (\neg y_{i,j} \vee x_j) \wedge (\neg x_i \vee \neg x_j \vee y_{i,j}) \right)$$

41 Transformation de Tseitin

- la technique de Tseitin sera particulièrement intéressante lorsqu'on devra mettre sous FNC des formules qui sont sous forme normale *disjonctive*

$$C_1 \vee C_2 \vee \cdots \vee C_n$$

où les C_i sont des conjonctions de littéraux.

- on va introduire une variable x_i pour chaque C_i, et on obtient la formule

$$(x_1 \vee x_2 \vee \cdots \vee x_n) \wedge (x_1 \leftrightarrow C_1) \wedge \cdots \wedge (x_n \leftrightarrow C_n)$$

- supposons que $C_i = l_1 \wedge \ldots l_k$ où les l_i sont des littéraux, alors mettre $x_i \leftrightarrow C_i$ sous FNC est assez simple :

$$x_i \leftrightarrow C_i \equiv (\bigwedge_{j=1}^{k} (\neg x_i \vee l_j)) \wedge (x_i \vee \bigvee_{j=1}^{k} \neg l_j)$$

qui est bien sous FNC.

42 A retenir

Principe (souvent) valable pour les SAT solveurs

Ne pas hésiter à introduire des nouvelles variables pour minimiser le nombre de clauses !

43 Variantes de SAT

- 2-SAT : les clauses ne contiennent qu'au plus deux littéraux.
  - Par exemple $(x_1 \vee \neg x_2) \wedge x_3 \wedge (x_4 \vee \neg x_1) \wedge (\neg x_3 \vee \neg x_2)$
  - 2-SAT est solvable en temps polynomial (alors que 3-SAT est NP-complet).

43 Variantes de SAT

- 2-SAT : les clauses ne contiennent qu'au plus deux littéraux.
  - Par exemple $(x_1 \vee \neg x_2) \wedge x_3 \wedge (x_4 \vee \neg x_1) \wedge (\neg x_3 \vee \neg x_2)$
  - 2-SAT est solvable en temps polynomial (alors que 3-SAT est NP-complet).
- QSAT : décider la satisfaisabilité de formules de la forme

$$\forall x_1 \exists q_1 \forall x_2 \exists q_2 \ldots \forall q_n \exists q_n . \phi$$

où ϕ est une formule en FNC construite sur les propositions $x_1, q_1, \ldots, x_n, q_n$.

43 Variantes de SAT

- 2-SAT : les clauses ne contiennent qu'au plus deux littéraux.
  - Par exemple $(x_1 \vee \neg x_2) \wedge x_3 \wedge (x_4 \vee \neg x_1) \wedge (\neg x_3 \vee \neg x_2)$
  - 2-SAT est solvable en temps polynomial (alors que 3-SAT est NP-complet).
- QSAT : décider la satisfaisabilité de formules de la forme

$$\forall x_1 \exists q_1 \forall x_2 \exists q_2 \ldots \forall q_n \exists q_n . \phi$$

où ϕ est une formule en FNC construite sur les propositions $x_1, q_1, \ldots, x_n, q_n$.

  - par exemple : $\forall x \exists q . (x \vee q) \wedge (\neg x \vee \neg q)$.

43 Variantes de SAT

- 2-SAT : les clauses ne contiennent qu'au plus deux littéraux.
  - Par exemple $(x_1 \vee \neg x_2) \wedge x_3 \wedge (x_4 \vee \neg x_1) \wedge (\neg x_3 \vee \neg x_2)$
  - 2-SAT est solvable en temps polynomial (alors que 3-SAT est NP-complet).
- QSAT : décider la satisfaisabilité de formules de la forme

$$\forall x_1 \exists q_1 \forall x_2 \exists q_2 \ldots \forall q_n \exists q_n . \phi$$

où ϕ est une formule en FNC construite sur les propositions $x_1, q_1, \ldots, x_n, q_n$.

  - par exemple : $\forall x \exists q . (x \vee q) \wedge (\neg x \vee \neg q)$.
  - Si p est vraie, alors q doit être fausse, et si p est fausse, alors q doit être vraie.

43 Variantes de SAT

- 2-SAT : les clauses ne contiennent qu'au plus deux littéraux.
  - Par exemple $(x_1 \vee \neg x_2) \wedge x_3 \wedge (x_4 \vee \neg x_1) \wedge (\neg x_3 \vee \neg x_2)$
  - 2-SAT est solvable en temps polynomial (alors que 3-SAT est NP-complet).
- QSAT : décider la satisfaisabilité de formules de la forme

$$\forall x_1 \exists q_1 \forall x_2 \exists q_2 \ldots \forall q_n \exists q_n . \phi$$

où ϕ est une formule en FNC construite sur les propositions $x_1, q_1, \ldots, x_n, q_n$.

  - par exemple : $\forall x \exists q.(x \vee q) \wedge (\neg x \vee \neg q)$.
  - Si p est vraie, alors q doit être fausse, et si p est fausse, alors q doit être vraie.
  - QSAT est PSPACE-complet (QSAT peut être résolu en espace polynomial et tous les problèmes pouvant être résolus en espace polynomial peuvent se réduire au problème QSAT en temps polynomial, autrement dit, QSAT est aussi difficile que tous les problèmes pouvant être résolu en espace polynomial)

44 Variantes de SAT

- MAX-SAT : étant donnée une formule ϕ en FNC et un entier $k \in \mathbb{N}$, peut-on satisfaire au moins k clauses ?
  - Par exemple $x \wedge y \wedge (\neg x \vee \neg y)$

44 Variantes de SAT

- MAX-SAT : étant donnée une formule ϕ en FNC et un entier $k \in \mathbb{N}$, peut-on satisfaire au moins k clauses ?
  - Par exemple $x \wedge y \wedge (\neg x \vee \neg y)$on peut satisfaire au plus deux clauses.

44 Variantes de SAT

- MAX-SAT : étant donnée une formule ϕ en FNC et un entier $k \in \mathbb{N}$, peut-on satisfaire au moins k clauses ?
  - Par exemple $x \wedge y \wedge (\neg x \vee \neg y)$on peut satisfaire au plus deux clauses.
  - Pourquoi MAX-SAT est aussi difficile que SAT ?

44 Variantes de SAT

- MAX-SAT : étant donnée une formule ϕ en FNC et un entier $k \in \mathbb{N}$, peut-on satisfaire au moins k clauses ?
  - Par exemple $x \wedge y \wedge (\neg x \vee \neg y)$on peut satisfaire au plus deux clauses.
  - Pourquoi MAX-SAT est aussi difficile que SAT ?
- WEIGHTED-MAX-SAT : on attribue des poids à chaque clause, on se donne un entier r, et on veut savoir s'il on peut satisfaire un ensemble de clauses dont la somme des poids est au moins r

44 Variantes de SAT

- MAX-SAT : étant donnée une formule ϕ en FNC et un entier $k \in \mathbb{N}$, peut-on satisfaire au moins k clauses ?
  - Par exemple $x \wedge y \wedge (\neg x \vee \neg y)$on peut satisfaire au plus deux clauses.
  - Pourquoi MAX-SAT est aussi difficile que SAT ?
- WEIGHTED-MAX-SAT : on attribue des poids à chaque clause, on se donne un entier r, et on veut savoir s'il on peut satisfaire un ensemble de clauses dont la somme des poids est au moins r
  - Par exemple $x \wedge y \wedge (\neg x \vee \neg y)$ avec $poids(x) = 1$, $poids(y) = 1$ et $poids(\neg x \vee \neg y) = 2$ et $r = 3$

44 Variantes de SAT

- MAX-SAT : étant donnée une formule ϕ en FNC et un entier $k \in \mathbb{N}$, peut-on satisfaire au moins k clauses ?
  - Par exemple $x \wedge y \wedge (\neg x \vee \neg y)$on peut satisfaire au plus deux clauses.
  - Pourquoi MAX-SAT est aussi difficile que SAT ?
- WEIGHTED-MAX-SAT : on attribue des poids à chaque clause, on se donne un entier r, et on veut savoir s'il on peut satisfaire un ensemble de clauses dont la somme des poids est au moins r
  - Par exemple $x \wedge y \wedge (\neg x \vee \neg y)$ avec $poids(x) = 1$, $poids(y) = 1$ et $poids(\neg x \vee \neg y) = 2$ et $r = 3$ En mettant x et y à vrai, on ne satisfait pas la dernière clause, et on obtient le poids total $1 + 1 = 2$. En mettant x à vrai et y à faux, on satisfait la première et la dernière clause, et on obtient le poids total $1 + 2 = 3$.

44 Variantes de SAT

- MAX-SAT : étant donnée une formule ϕ en FNC et un entier $k \in \mathbb{N}$, peut-on satisfaire au moins k clauses ?
  - Par exemple $x \wedge y \wedge (\neg x \vee \neg y)$on peut satisfaire au plus deux clauses.
  - Pourquoi MAX-SAT est aussi difficile que SAT ?
- WEIGHTED-MAX-SAT : on attribue des poids à chaque clause, on se donne un entier r, et on veut savoir s'il on peut satisfaire un ensemble de clauses dont la somme des poids est au moins r
  - Par exemple $x \wedge y \wedge (\neg x \vee \neg y)$ avec $poids(x) = 1$, $poids(y) = 1$ et $poids(\neg x \vee \neg y) = 2$ et $r = 3$ En mettant x et y à vrai, on ne satisfait pas la dernière clause, et on obtient le poids total $1 + 1 = 2$. En mettant x à vrai et y à faux, on satisfait la première et la dernière clause, et on obtient le poids total $1 + 2 = 3$.
  - Pourquoi WEIGHTED-MAX-SAT est aussi difficile que MAX-SAT ?

44 Variantes de SAT

- MAX-SAT : étant donnée une formule ϕ en FNC et un entier $k \in \mathbb{N}$, peut-on satisfaire au moins k clauses ?
  - Par exemple $x \wedge y \wedge (\neg x \vee \neg y)$on peut satisfaire au plus deux clauses.
  - Pourquoi MAX-SAT est aussi difficile que SAT ?
- WEIGHTED-MAX-SAT : on attribue des poids à chaque clause, on se donne un entier r, et on veut savoir s'il on peut satisfaire un ensemble de clauses dont la somme des poids est au moins r
  - Par exemple $x \wedge y \wedge (\neg x \vee \neg y)$ avec $poids(x) = 1$, $poids(y) = 1$ et $poids(\neg x \vee \neg y) = 2$ et $r = 3$ En mettant x et y à vrai, on ne satisfait pas la dernière clause, et on obtient le poids total $1 + 1 = 2$. En mettant x à vrai et y à faux, on satisfait la première et la dernière clause, et on obtient le poids total $1 + 2 = 3$.
  - Pourquoi WEIGHTED-MAX-SAT est aussi difficile que MAX-SAT ? MAX-SAT est le cas particulier de WEIGHTED-MAX-SAT où le poids de chaque clause vaut 1.

45 Références

- http://en.wikipedia.org/wiki/Boolean_satisfiability_problem
- cours/tutoriaux : http://www.cse.unsw.edu.au/~tw/sat/
- http://www.pps.jussieu.fr/~treinen/teaching/ (cours de Ralf Treinen)
- http://www.cs.wm.edu/~idillig/cs780-02/lecture3.pdf
- Handbook of satisfiability. A. Biere, M. Heule, H. van Maaren, T. Walsh. IOS Press. 2008. 981 pages.